# HOW SQUALIFY WORKS

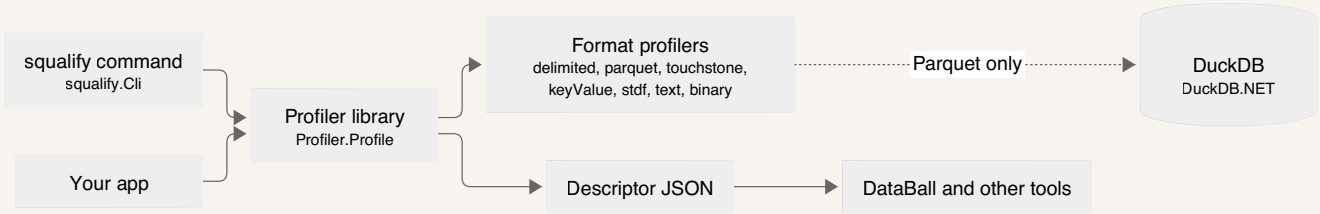
> **Design stage.** This page describes the agreed design (spec pass, 2026-10-13, amended after the sample-data review the same day). These slices have landed. sq-01 added the CLI shell, `ToolInfo` and `SqualifyException`. sq-02 added the descriptor model, the canonical JSON writer and the determinism kit (`DetMath`, `XxHash64`, `Ordinal`) and the golden-file test harness. sq-03 added the byte pass, the format registry and the generic text and binary profiles, so `squalify profile` now works on any file. sq-16 added the value classifier (`ValueClassifier`), which classifies one value as empty, true/false, a number, a date or time, or text. sq-04 added the delimited-text profiler: delimiter and quoting, preamble, annotation, header, units, data and footer regions, padding, the decimal separator and per-field type votes. sq-05 added the field accumulators (`FieldAccumulator`), which turn those votes into full field profiles: the type, null tokens, number and date statistics, lengths, shape masks and the constant and unique flags, in bounded memory. sq-14 added the header-unit conventions (`HeaderConventionParser`, `UnitTable`): each header name is split into a base name and a unit, and the file reports its dominant convention. Every source file that does not exist yet is marked *(planned)*. When an architectural review or a slice changes how squalify works, this page and its diagrams are updated in the same change (see “Docs” in `AGENTS.md`).

squalify reads one data file and writes a **descriptor**: a JSON summary of what the file is and what is inside it. Other tools, such as DataBall, read the descriptor to decide how to handle the file. squalify never changes the file and never needs the network.

Each diagram shows one idea. Labels are in plain language, with the code name in small text. Source paths are relative to `src/squalify/`, so the CLI project is written `../squalify.Cli/`.

## 1. THE PIECES

The `squalify` command is a thin shell around the library. Each kind of file has its own profiler. DuckDB is used only to read Parquet. The descriptor is the only thing that leaves the tool.

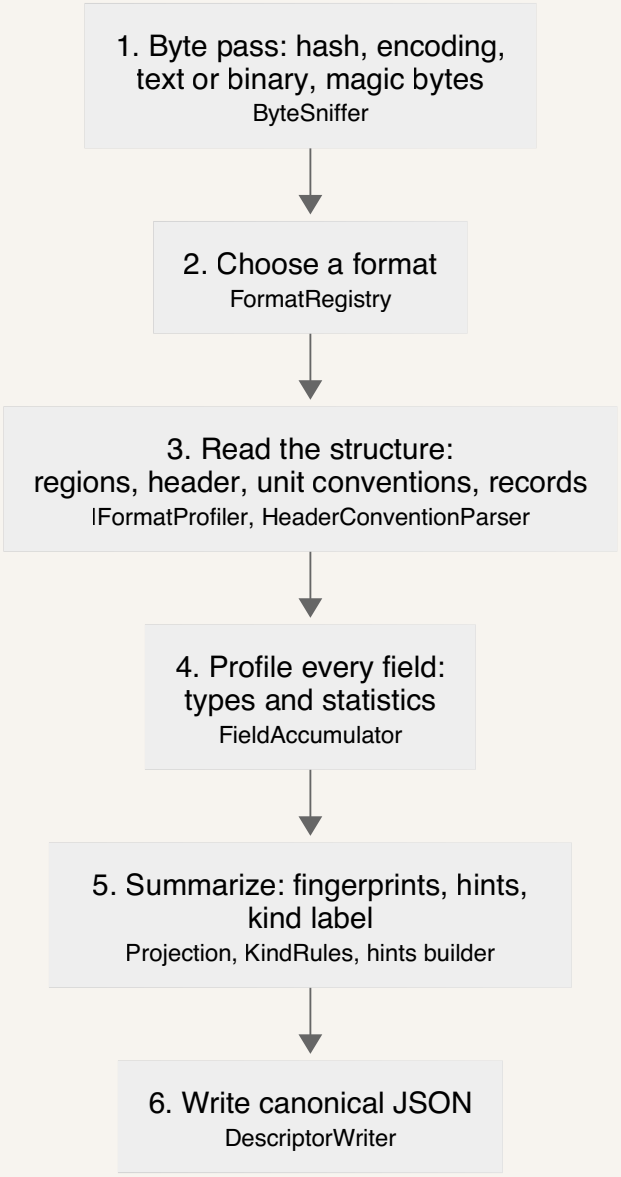



*Source:* `../squalify.Cli/CommandFactory.cs`, `../squalify.Cli/CliApp.cs`, `ToolInfo.cs`, `SqualifyException.cs`, `Descriptor/`, `Json/DescriptorWriter.cs`, `Profiler.cs`, `Formats/FormatRegistry.cs`, `Formats/GenericTextProfiler.cs`, `Formats/GenericBinaryProfiler.cs`, `Delimited/DelimitedProfiler.cs`; *the other profilers (Parquet, Touchstone, key=value, STDF) under* `Formats/`

*(planned).* The CLI works today: `squalify --version` prints the tool version, and `squalify profile <file>` writes the descriptor to stdout, or to the `-o` file. The `delimited`, `text` and `binary` profilers exist so far, so every file is described as a delimited table, text, binary or empty; the other profilers in the diagram are planned. A directory is an error.

## 2. ONE PROFILE RUN

Every file goes through the same steps. A step that reads the file reads all of it, start to end, and keeps only bounded summaries in memory. The file can be far larger than RAM.

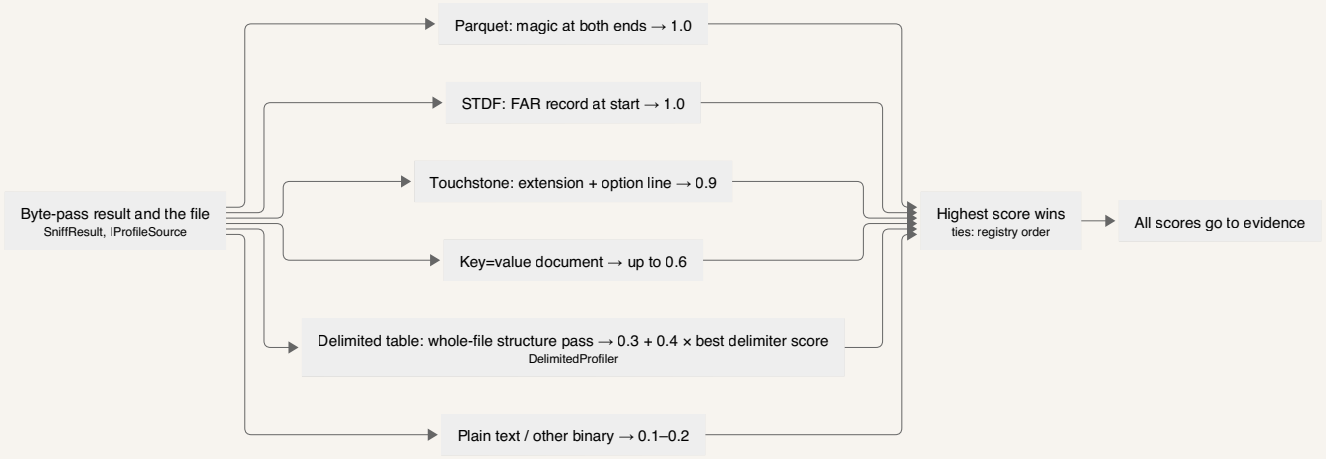



*Source:* `Json/DescriptorWriter.cs`, `Profiler.cs`, `Sniff/ByteSniffer.cs`, `Formats/FormatRegistry.cs`, `Formats/IFormatProfiler.cs`, `Delimited/DelimitedProfiler.cs`, `Units/HeaderConventionParser.cs`; `Fields/FieldAccumulator.cs`, `Delimited/DelimitedPass.cs`; `Fingerprint/`, `Kind/` *(planned).* Steps 3 to 5 are only partly built: the delimited, generic text and generic binary profilers make up step 3 for now (the

delimited one also parses the header-unit conventions here, section 8), step 4 is built for delimited text (the other formats will feed the same field accumulators), and the fingerprint and kind label hold fixed placeholder values until their slices land. The row classes are still to come. For delimited text, step 3 is two reads of the file: one while the format is chosen (the structure pass) and one when the delimited profiler has won.

## 3. CHOOSING A FORMAT

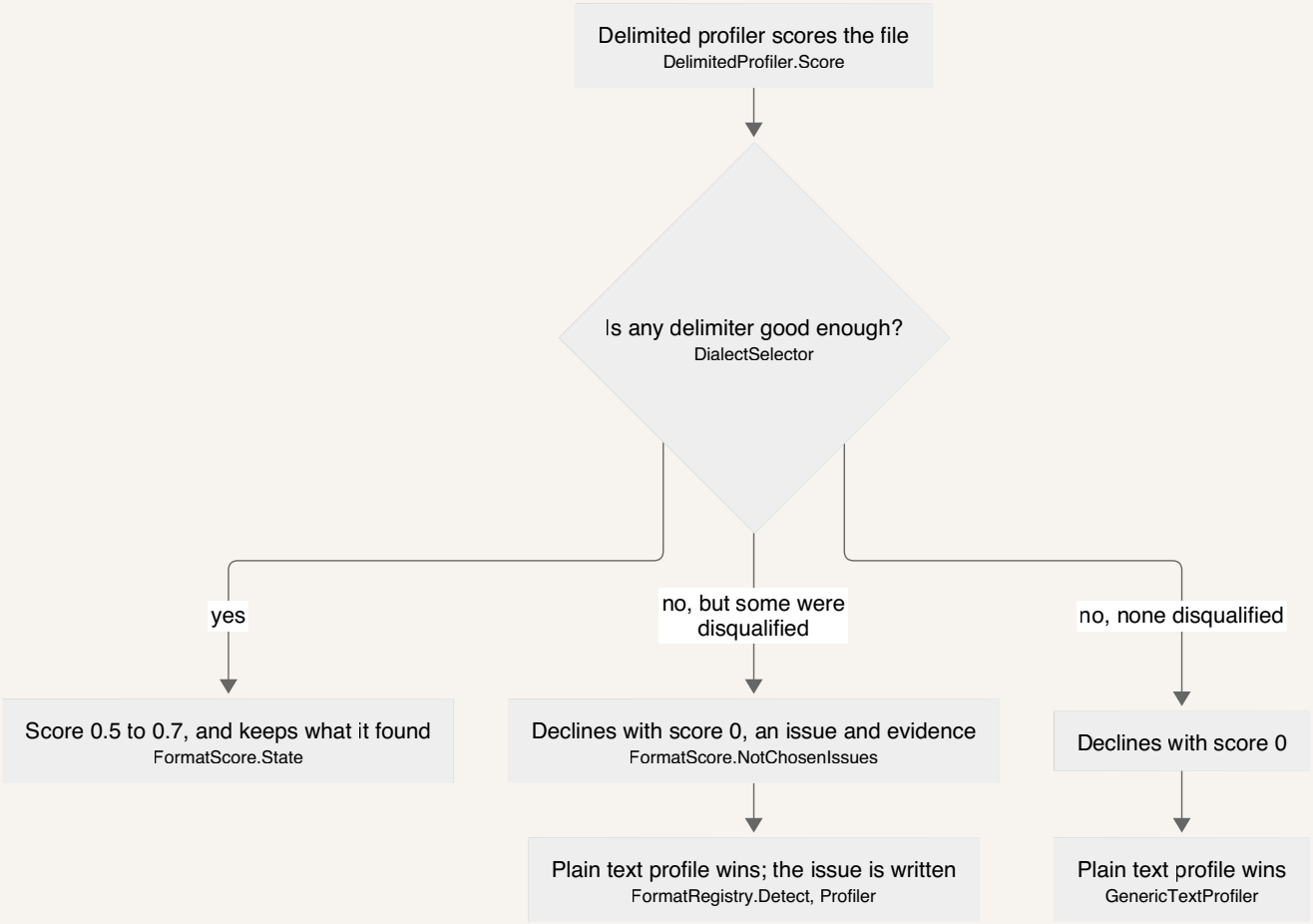
Every profiler looks at the file and gives a score from 0 to 1. The highest score wins, and a tie goes to whichever profiler comes first in the registry list. Every score and its reason are kept in the descriptor's evidence, so you can see why a format was chosen. Most profilers score from the byte-pass result alone. A delimited table cannot be recognized that way, so the delimited profiler is handed the file too and reads all of it while it scores (the structure pass of section 4); if it wins, it keeps what it found, so the file is not read again for the same work.




*Source:* `Formats/FormatRegistry.cs`, `Formats/IFormatProfiler.cs`, `Delimited/DelimitedProfiler.cs`, `Formats/GenericTextProfiler.cs`, `Formats/GenericBinaryProfiler.cs`*; the other profilers are planned.* A file with no bytes is called empty before any profiler scores it. For files that are not empty, the evidence lists the score of each profiler that is registered, in registry order; today that is delimited, text and binary.

A profiler can also decline a file and say why. When the delimited profiler finds delimiter evidence but gives up, because no delimiter is good enough and at least one candidate was disqualified, it scores 0 and hands

back an issue (`tooIrregular`) and its evidence. The registry passes these on even though the profiler lost, so the plain text profile describes the file and the descriptor still explains why it is not treated as a table.

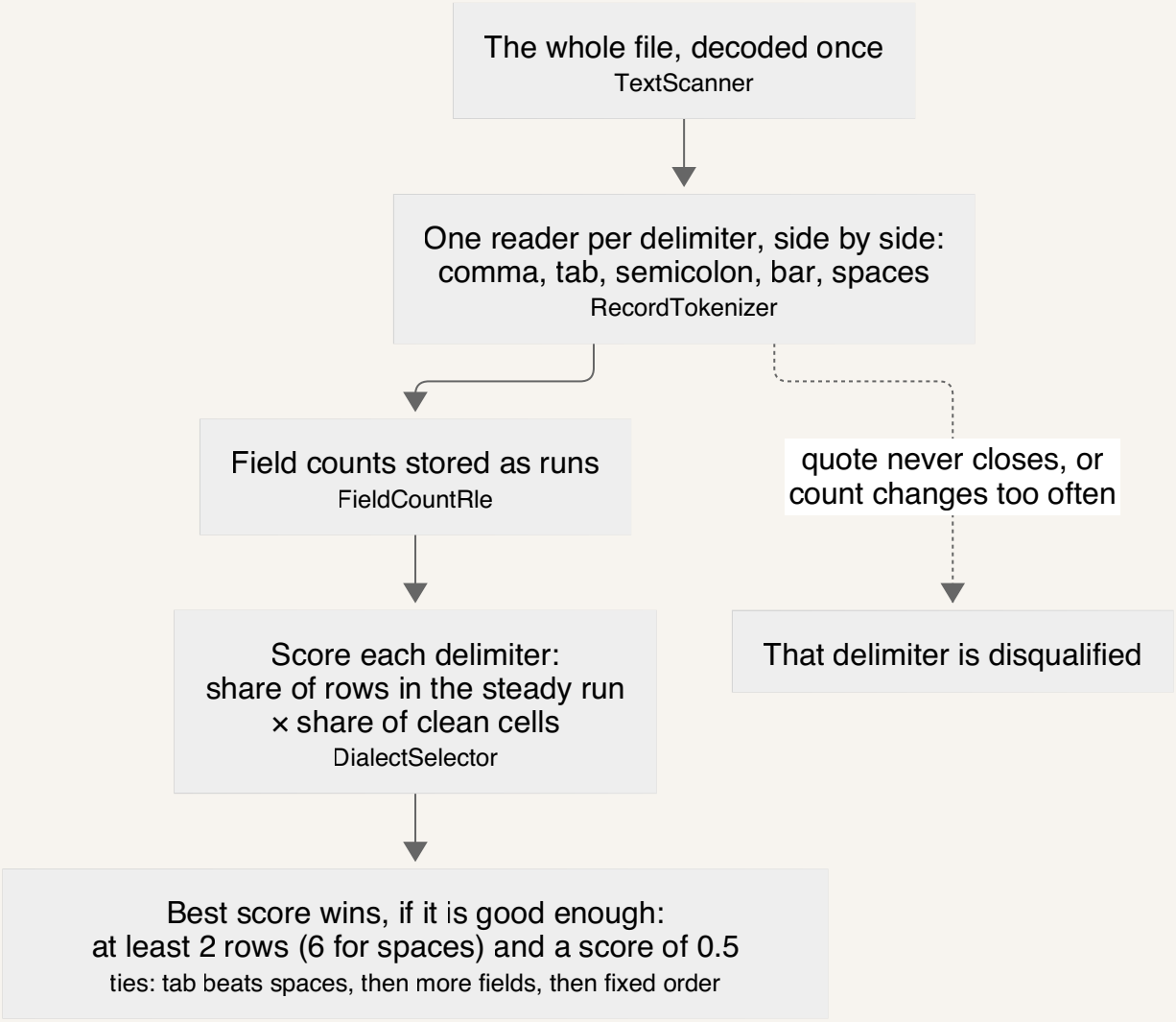



*Source:* `Delimited/DelimitedProfiler.cs`, `Delimited/DialectSelector.cs`, `Formats/IFormatProfiler.cs`, `Formats/FormatRegistry.cs`, `Profiler.cs`.

## 4. READING A DELIMITED FILE

A lab CSV is often more than a header and rows. squalify splits the file into regions. It tries every common delimiter at the same time, in one pass over the whole file. It picks the delimiter that gives the longest

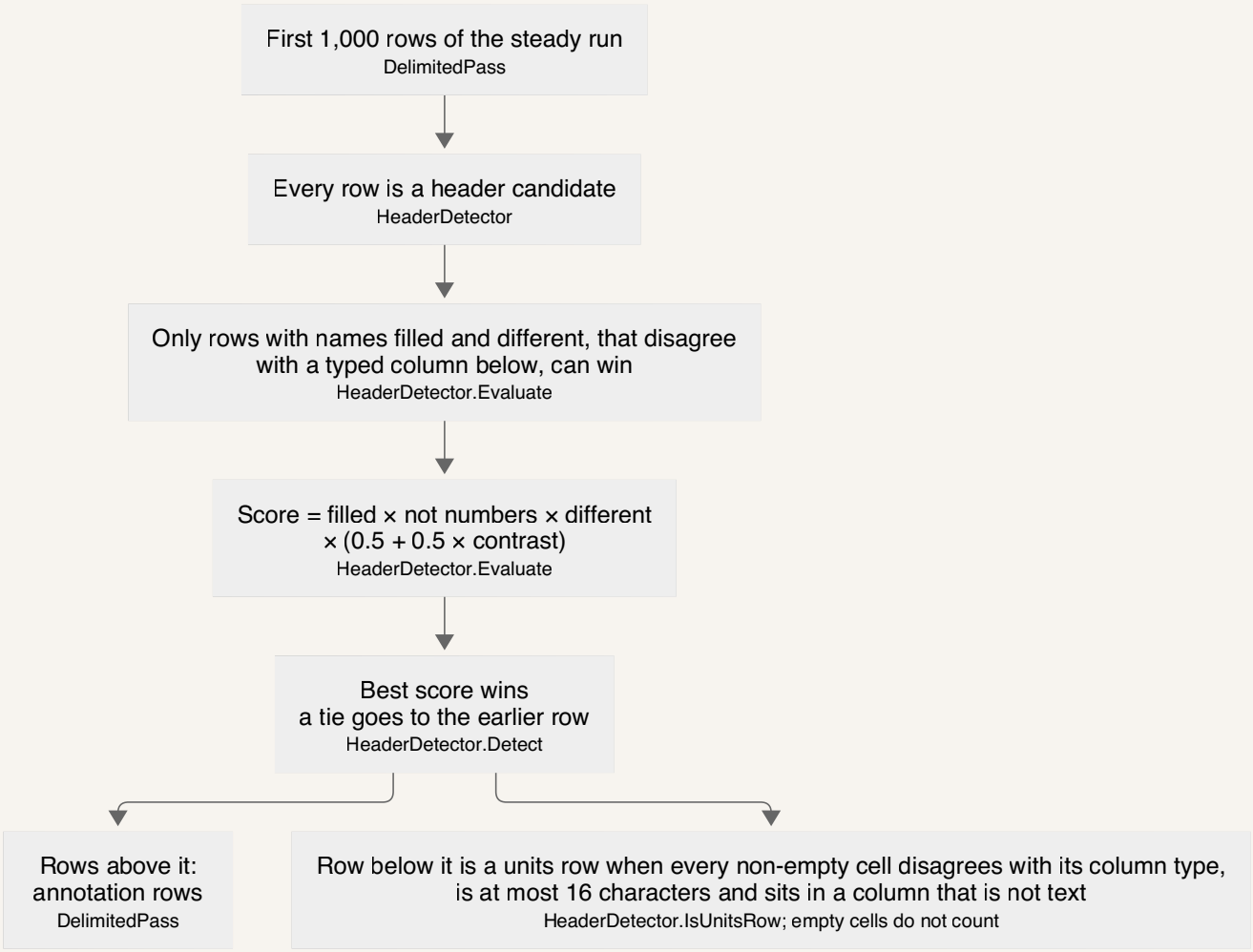
steady run of rows with the same field count, and whose cells split cleanly. On tables padded with spaces, splitting on spaces looks almost as steady, so cleanliness decides.




*Source:* `Delimited/TextScanner.cs`, `Delimited/RecordTokenizer.cs`, `Delimited/FieldCountRle.cs`, `Delimited/DialectSelector.cs`. Quotes follow the usual CSV rule (a doubled quote is one quote, and a quoted cell may hold a comma or a line break). A closing quote may be followed by spaces before the comma or the line end, as in padded tables. A stray quote inside a quoted cell, with no comma or line end after it, is kept as plain text, because some instruments write quotes inside a quoted cell without doubling them. A cell counts as clean when it is quoted or contains none of the characters `,` `;` `|` other than its own delimiter, and, for those three delimiters, no tab inside the cell. Spaces never make a cell unclean, and a decimal comma such as `1,5` does not count, so a semicolon file with decimal commas is told apart from a comma file. A delimiter is only chosen when it is good enough: its steady run has at least 2 rows (6 for spaces) and its score is at least 0.5. When tab and spaces tie, tab wins.

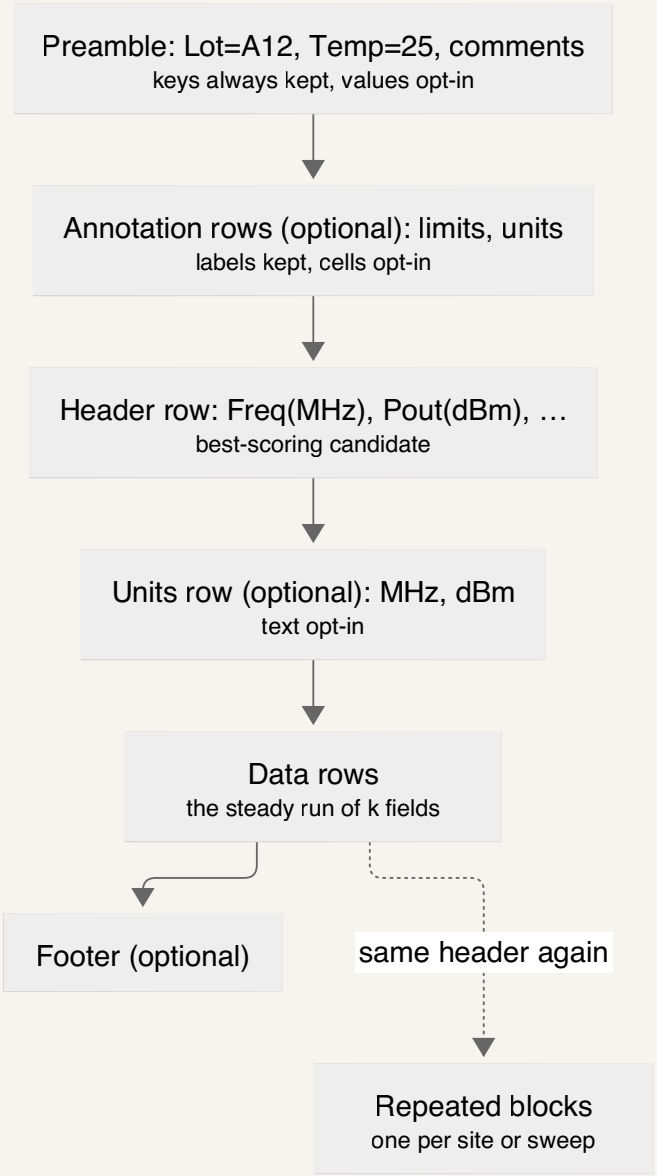
The header is not always the first row of that run. Some exports put rows of limits, units or line numbers above the header, lined up with its columns. squalify scores the first rows of the run and picks the one that

looks most like names over typed data. The rows above it become **annotation** rows: their labels are kept, their cells are opt-in.




*Source:* `Delimited/DelimitedPass.cs`, `Delimited/HeaderDetector.cs`. The contrast is the share of typed columns (columns that are not text) that the candidate disagrees with, so a row of units scores below the header that sits under it. A header with a blank or repeated name is still taken when nothing else qualifies, with an issue. A table made only of text takes its first row as the header and says so in the evidence. A first data row that is all `NULL` is never taken for a units row, so the interleaved separator rows of a DangerTest export do no harm. A comment line (`#`, `!`, `//`) is free text: it is never a header, an annotation row or a label, wherever it sits. The evidence lists the eligible candidates and the winner, and one summary line that counts the rest by reason.

With the dialect and header known, the file falls into regions, each with line and byte ranges. Regions tile the file: a blank line belongs to the region before it.




*Source:* `Delimited/DelimitedPass.cs`, `Delimited/LineClassifier.cs`, `Delimited/DialectSelector.cs`, `Delimited/HeaderDetector.cs`; `Delimited/BlockDetector.cs` *(planned).*

squalify also reports how the table is padded (fixed line length or not) and whether numbers use a decimal comma. It describes a fixed-width table but still reads every row through the tokenizer. Every data row is read a second time and handed to the field accumulators, one per column, so the descriptor says what each field holds; section 5 shows what they keep.

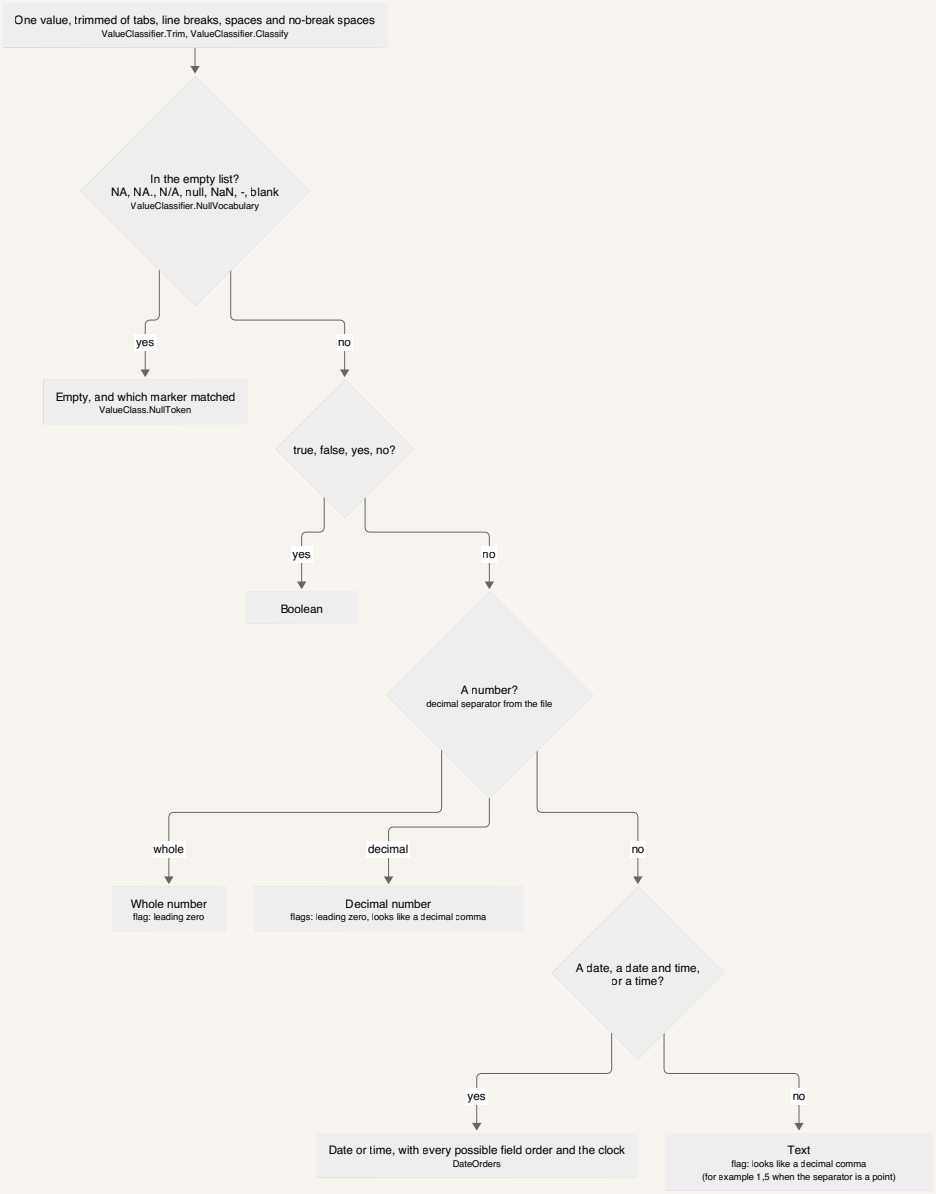
## 5. PROFILING ONE FIELD

Each value is classified on its own: empty, true/false, whole number, decimal number, date, time or text. A field's type is the narrowest type that fits **every** non-empty value. Empty markers such as `NA`, `NA.` or `–`

come from a fixed, versioned list that is never learned from the data, and the classifier says which marker it matched, so a consumer can decide which markers it treats as empty. The descriptor counts the markers by their entry in that list, so `NULL`, `Null` and `null` are all counted as `null`.

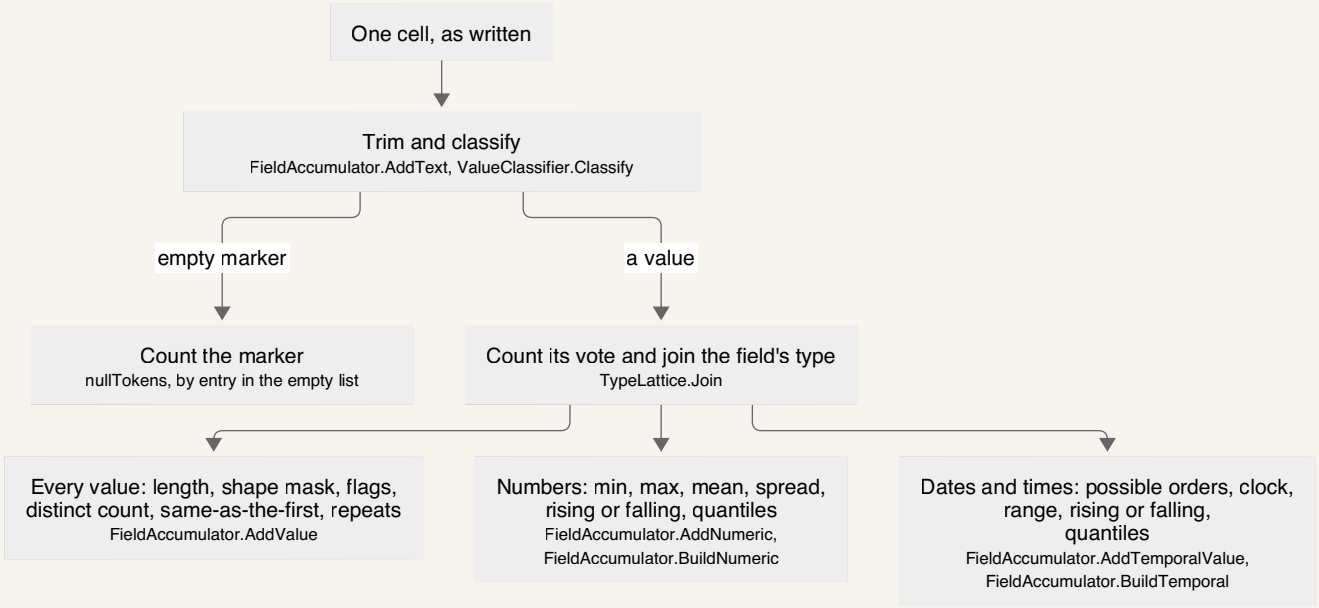
Dates never assume a locale. `6/5/2026` could be June 5 or May 6, so squalify reports both possible orders and marks the field ambiguous. It then leaves out the date range and whether the dates rise or fall, because both would assume one order. The same goes for the clock: a time with `AM` or `PM` is a 12-hour time, and a two-digit year such as `6/5/26` stays text, because squalify would have to guess the century.

The classifier looks at one value at a time and takes the first test that fits, in this order. Two flags ride along: *leading zero* only on whole and decimal numbers, and *looks like a decimal comma* on a decimal number or text. It also hands on what it parsed (the number, or the date and time parts), so nothing parses the text a second time. It only reads the value and the file's decimal separator, so it gives the same answer on every machine and in every language setting.



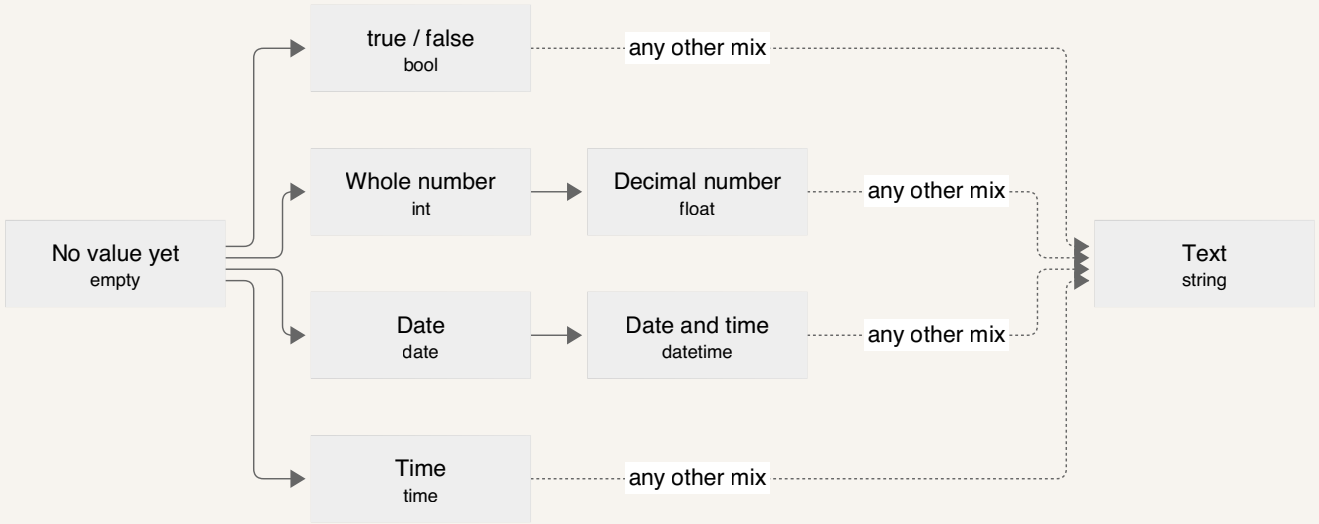
*Source:* `Values/ValueClassifier.cs`, `Values/DateOrders.cs`, `Values/ValueClass.cs`.

Every column has its own **field accumulator**. It receives the column's values in file order, in one pass, and keeps only bounded summaries, so a file can be far larger than memory. The delimited reader feeds it the text of each cell; other formats will feed it typed values through the same class, so the statistics match whatever the format.



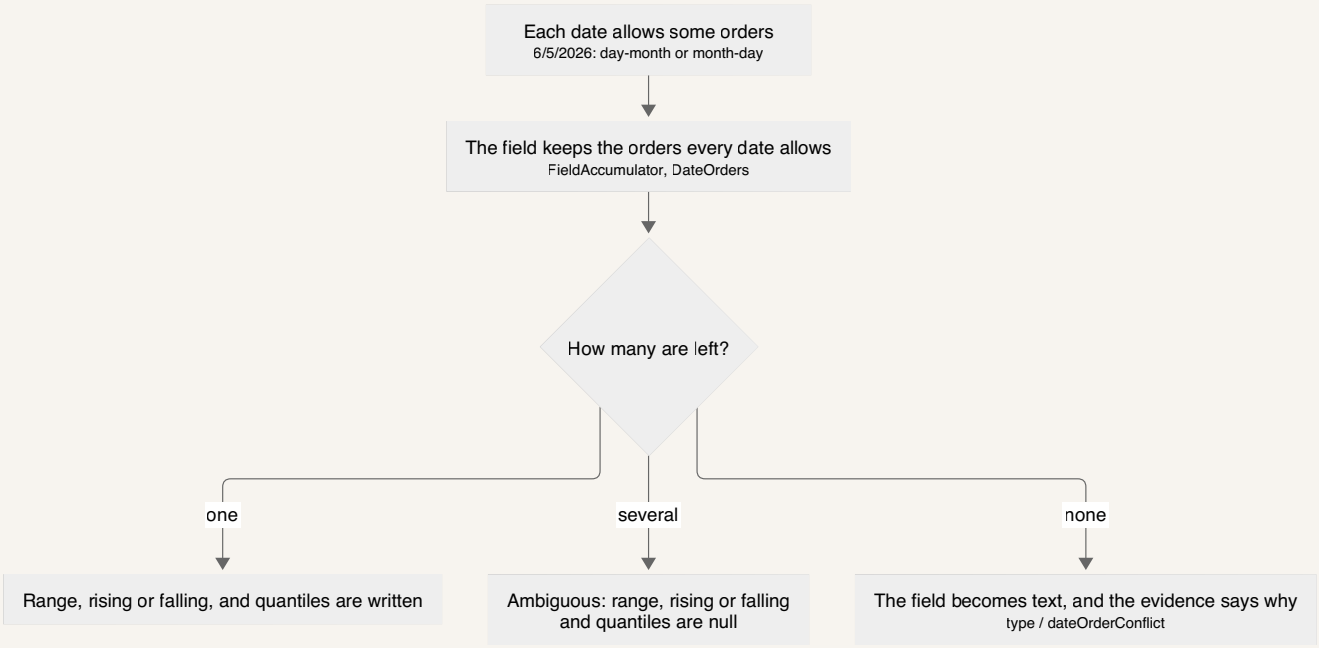

*Source:* `Fields/FieldAccumulator.cs`, `Fields/TypeLattice.cs`, `Fields/TemporalMath.cs`, `Delimited/DelimitedPass.cs` *(the hook is* `DelimitedPass.ProcessData`*).*

A field's type is the join of what its values vote for. Whole numbers join with decimal numbers, a date joins with a date and time, and any other mix is text. A value that is empty does not vote.




*Source:* `Fields/TypeLattice.cs` *(the join is* `TypeLattice.Join`*).*

Dates are the one place where the type can depend on more than the votes. Each date allows some field orders, and the field keeps the orders that every one of its dates allows. The votes are still reported as they were counted.

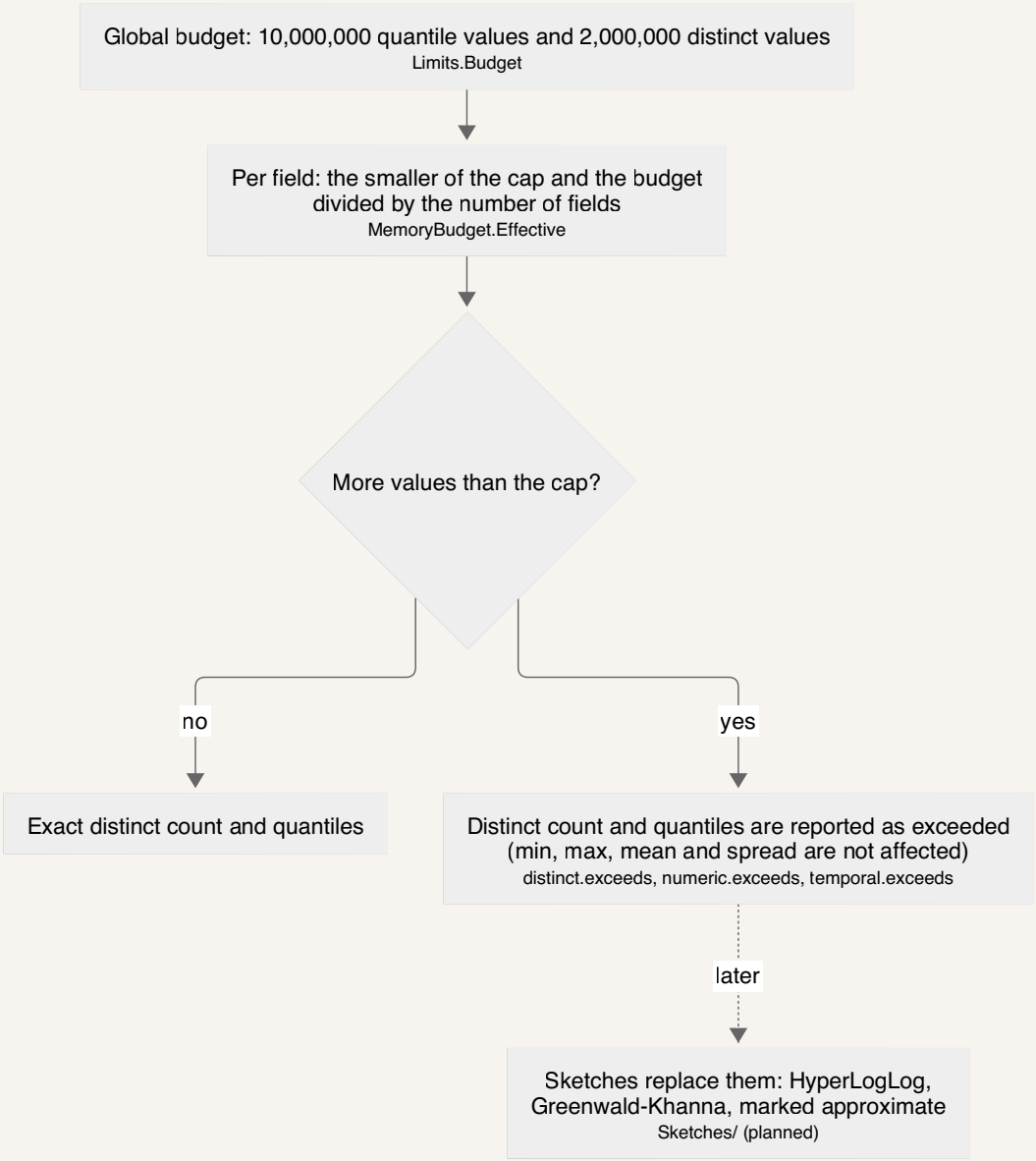



*Source:* `Fields/FieldAccumulator.cs`, `Values/DateOrders.cs`, `Fields/TemporalMath.cs`.

Every value of a field is also reduced to a **shape mask**: `9` for a digit, `A` for an upper-case letter, `a` for a lower-case letter, punctuation and spaces kept as they are, and `?` for anything that is not plain ASCII (and for curly braces, which the run counts use). A run of two or more is written with a count, so `Lot-A12` is `Aa{2}-A9{2}`. The masks show how a field is written without showing what is in it. A field keeps at most 64 masks and counts the rest as `other`. File names use the same masks.

Some facts about a field are cheap to keep exactly, and some are not. Counts, the minimum and maximum, the mean and the spread take constant memory. The distinct count and the quantiles need the values themselves, so they are exact only up to a cap (10,000 distinct values and 100,000 values per field). A global budget keeps wide files bounded: each field gets the smaller of its cap and its share of the budget, so a file of 2,000 columns gets 5,000 quantile values and 1,000 distinct values per column, and `limits` in

the descriptor records the caps in effect. Past a cap, the distinct count and the quantiles are reported as exceeded until sketches replace them.



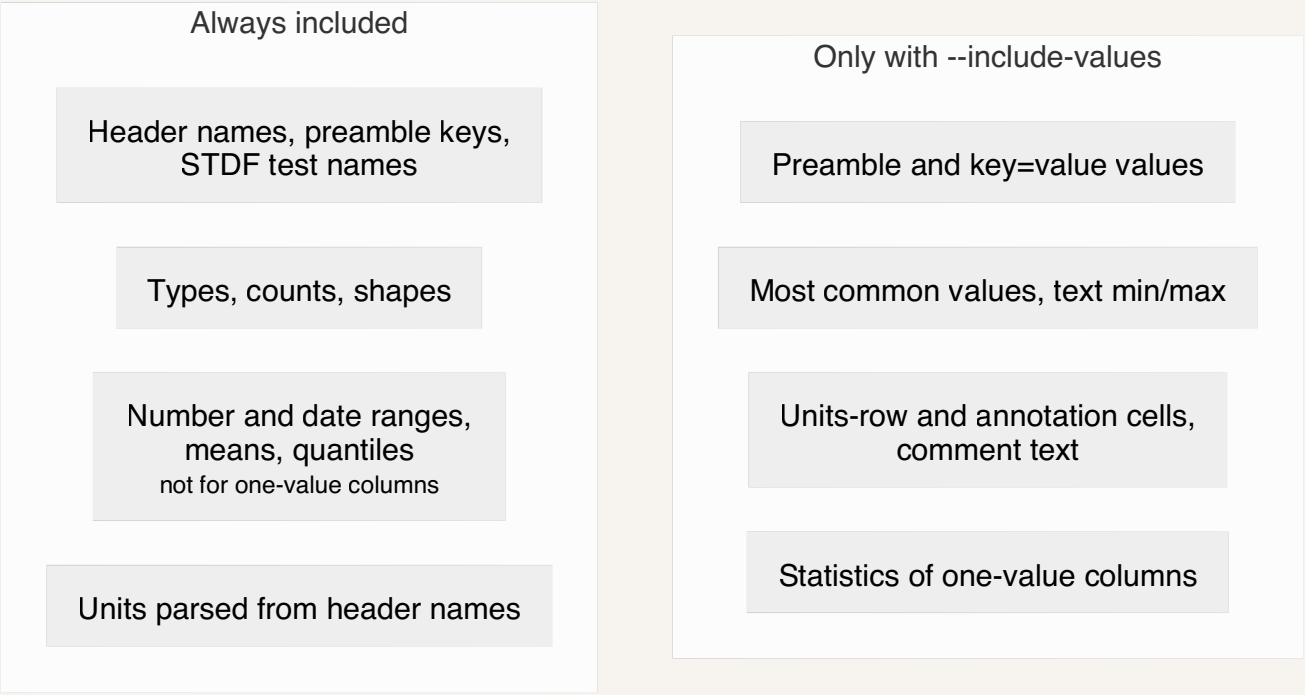

*Source:* `Fields/MemoryBudget.cs`, `Fields/ProfileLimits.cs`, `Fields/FieldAccumulator.cs`, `Descriptor/Options.cs`; `Sketches/` *(planned).*

## 6. WHAT IS ALWAYS INCLUDED, AND WHAT NEEDS `--include-values`

By default a descriptor describes the **shape** of the data and its statistics, not its contents. Literal text from the file appears only when you ask for it: all of it with `--include-values`, or only named keys and columns with `--include-values-for <name>`.

A column with one value for the whole file, such as a serial number, is a special case. Its range would just be that value, so its statistics are opt-in too: the descriptor leaves them out and says `statsWithheld: "constant"`, but still reports the column's length, shapes and distinct count. A column that has one value

and otherwise only empty markers counts as one-value, and for numbers and dates “one value” means the same number or date however it is written, so `1`, `1.0` and `01` are one value.

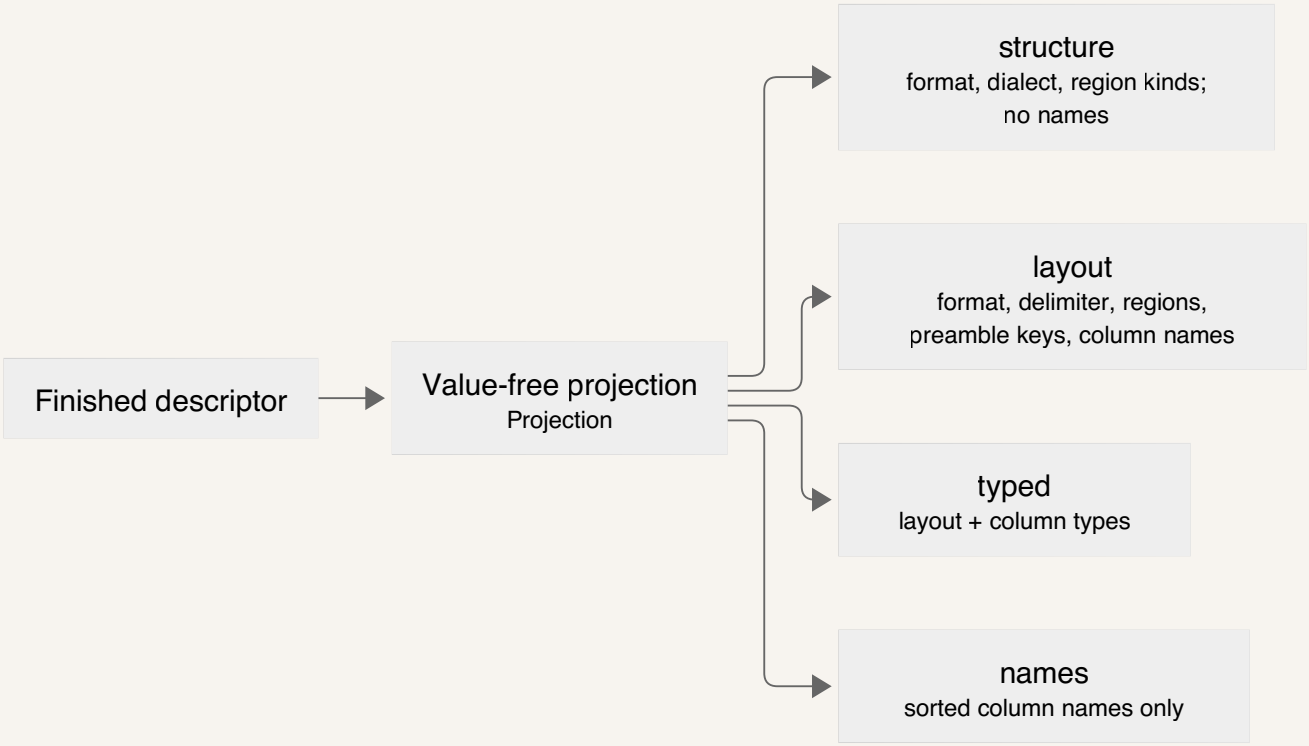



*Source: `Descriptor/Options.cs` (`ProfileOptions`), `Delimited/DelimitedPass.cs` (`DelimitedPass.Allows` decides, field by field and key by key, what may be written), `Fields/FieldAccumulator.cs` (the one-value-column rule). One `ValuesPolicy` that lists every value-bearing member in one place is planned.*

## 7. FINGERPRINTS

squalify hashes a stripped-down copy of the descriptor that contains no data values. Two files of the same type therefore get the same fingerprint, even when their data differs. There are four fingerprints at different levels of strictness. `structure` uses no column names at all, so it survives a changed column set.

A matching fingerprint does not prove two files are the same kind of file. Many unrelated CSVs share one `structure`, and files whose columns change per run get a new `layout` each time. Consumers combine fingerprints with names and hints.

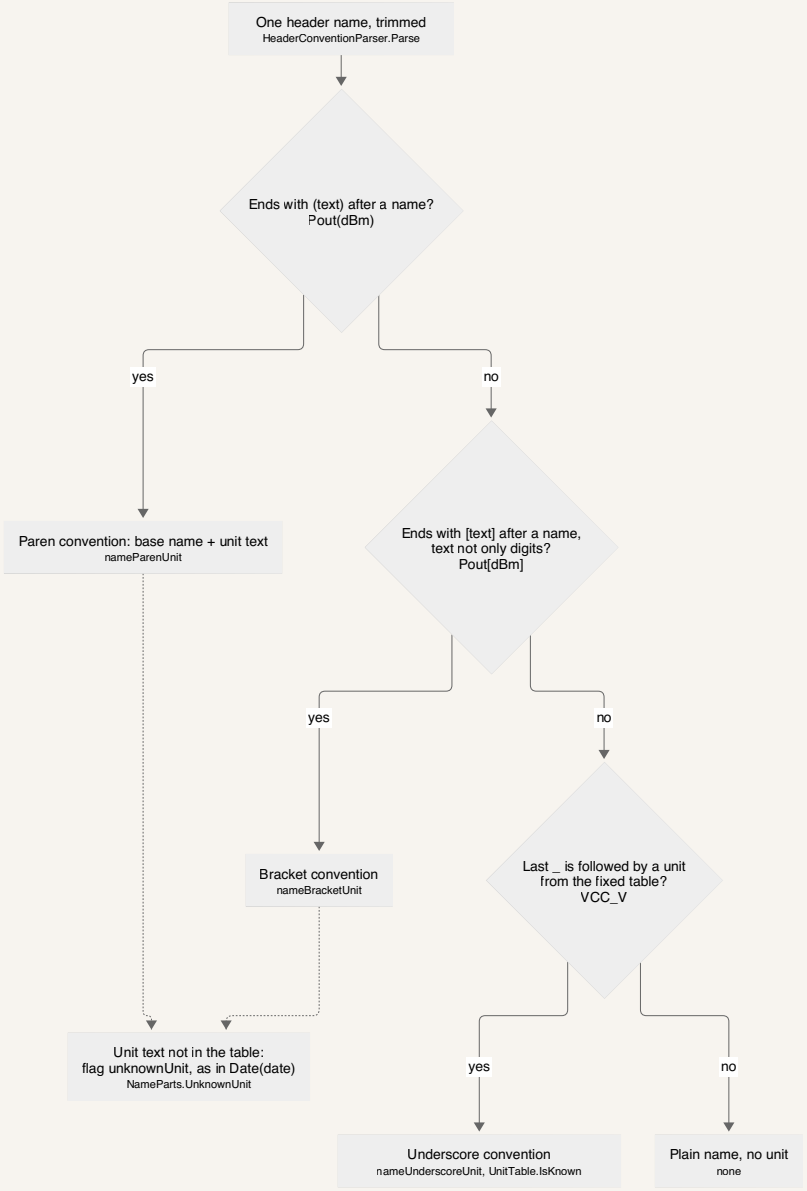



*Source:* `Fingerprint/Projection.cs` *(planned).*

## 8. HEADER-UNIT CONVENTIONS AND THE KIND LABEL

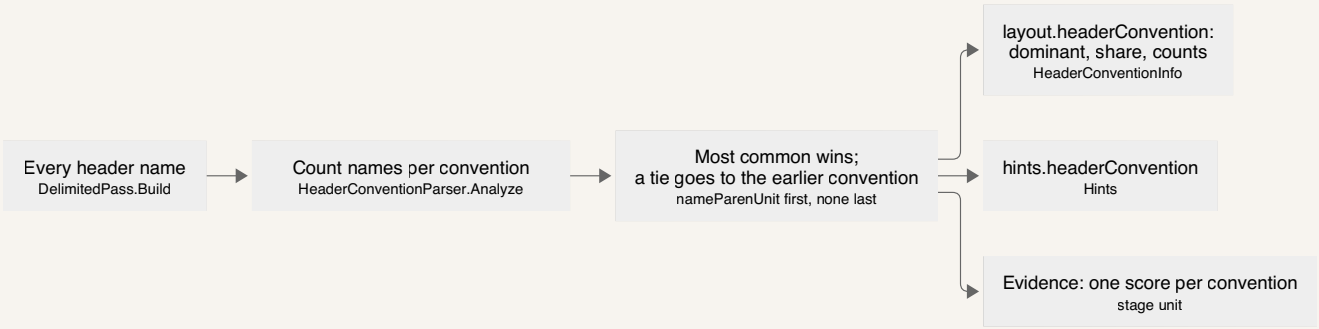
Lab exports carry units in many custom ways: `I_Total(A)`, `Pout[dBm]`, `VCC_V`, a units row under the header, or not at all. squalify reports which way each column name does it, and which way the file does it most. It reads the header names only, never the values, so the result is always included in the descriptor.

Each header name is tried against a short list, in this order, and the first match wins. The patterns and the unit table follow DataBall's header parser, so a consumer can map the conventions directly. The one addition is the bracket form, and digits alone in brackets (`Trace[0]`) are an array index, not a unit.



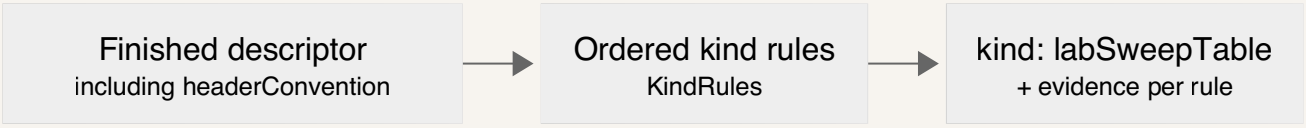

*Source:* *`Units/HeaderConventionParser.cs`*, *`Units/UnitTable.cs`*. The parenthesis and bracket forms accept any unit text and flag the ones the table does not know, so `RowID(id)` is split into the base `RowID` and the unit `id`, with the flag. The underscore form needs a known unit, so `I_Total` stays a plain name. The table is fixed and versioned (`limits.unitTable`): V, A, W, Hz, s, Ohm, F, H, dB, dBm, dBc, %, C, K and the angle units `°` and `deg`, with the SI prefixes f to G on the first eight. Matching is case-sensitive, except for a short list of spelled-out aliases such as `ohm` and `degC`. When the file has a units row, every name without a unit of its own gets the convention `unitsRow`, but only when its units cell has a value; the units text itself is a value and stays opt-in.

The file then gets one summary: how many names use each convention, and the most common one. A tie goes to the earlier convention in the list above, with `none` last, so a file with as many unit names as plain names counts as using the unit convention. A table with no header has no names, so it gets no convention.




*Source:* `Units/HeaderConventionParser.cs`, `Delimited/DelimitedPass.cs`, `Profiler.cs`.

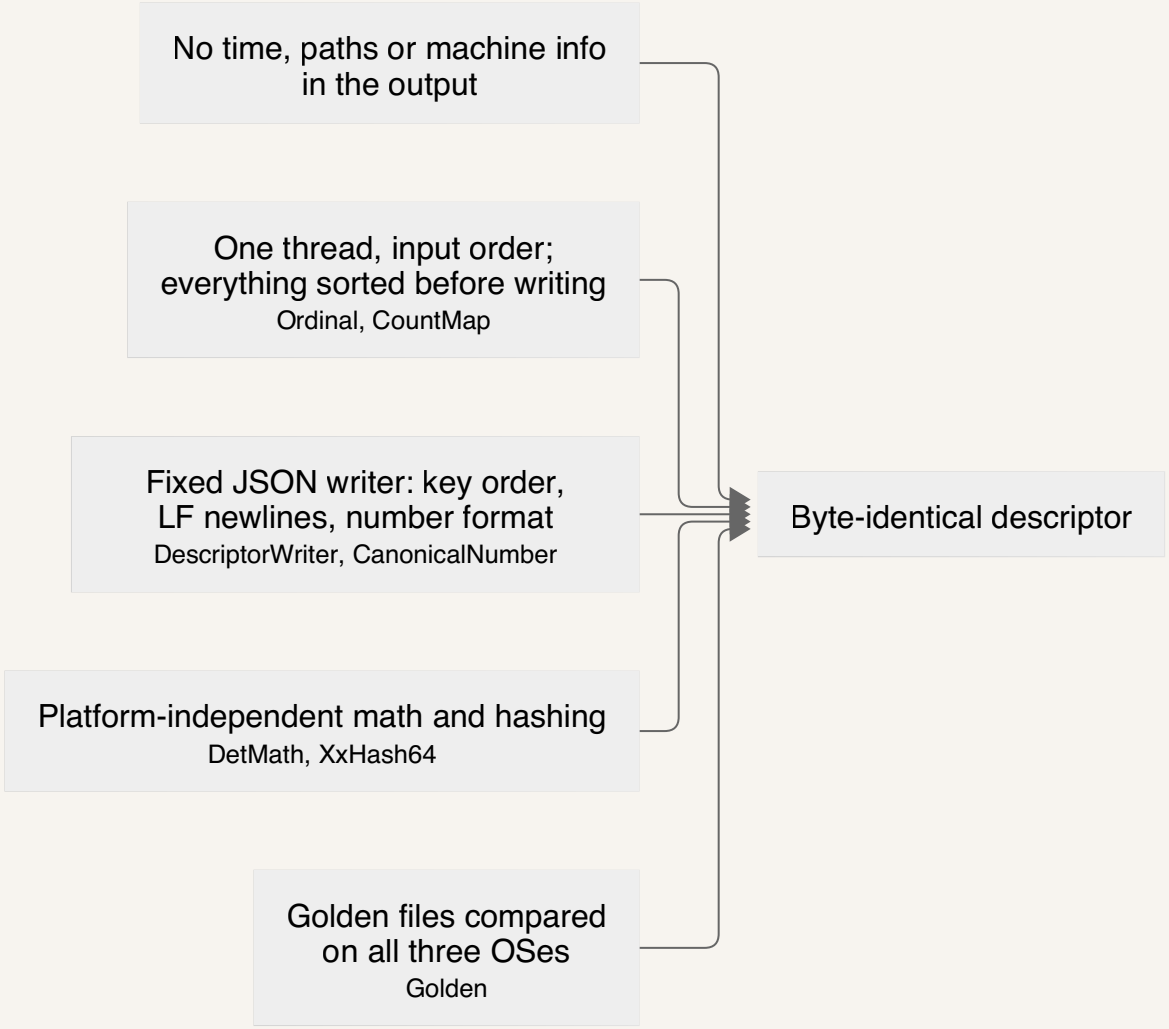
A second rule set, the **kind label**, is still planned. It will read the finished descriptor (never the file) and suggest a generic label such as `labSweepTable`, `stdfTestLog` or `touchstoneNetwork`. The first rule in a fixed, ordered list that matches wins, and every rule's result is kept as evidence.




*Source:* `Kind/` *(planned).*

## 9. WHY THE OUTPUT NEVER CHANGES

The same file and the same squalify version always give the same descriptor, byte for byte, on Windows, macOS and Linux. Five guardrails make that true.

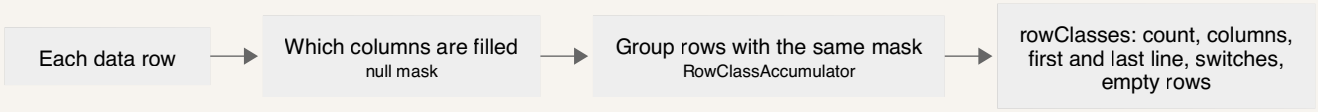



*Source:* `Determinism/Ordinal.cs`, `Determinism/DetMath.cs`, `Determinism/XxHash64.cs`, `Descriptor/CountMap.cs`, `Json/DescriptorWriter.cs`, `Json/CanonicalNumber.cs`; `../../tests/squalify.Tests/Golden.cs`.

Logarithms are the one place where platforms can disagree: the system math library may differ by one bit between operating systems. squalify therefore computes them itself, from additions, multiplications and divisions only, which every CPU rounds identically. A reference table made with an arbitrary-precision tool pins the result bit for bit. Hashing uses squalify's own copy of xxHash64, checked against published test vectors.

## 10. ROW CLASSES

Some exports mix several kinds of row in one table: measurement rows, setup rows, error rows and empty separator rows, switching kind on almost every line. squalify groups rows by which columns are filled in and reports each group, how often the kind switches, and how many rows are empty. Column null counts are also given without the empty rows, so separators do not look like missing measurements.

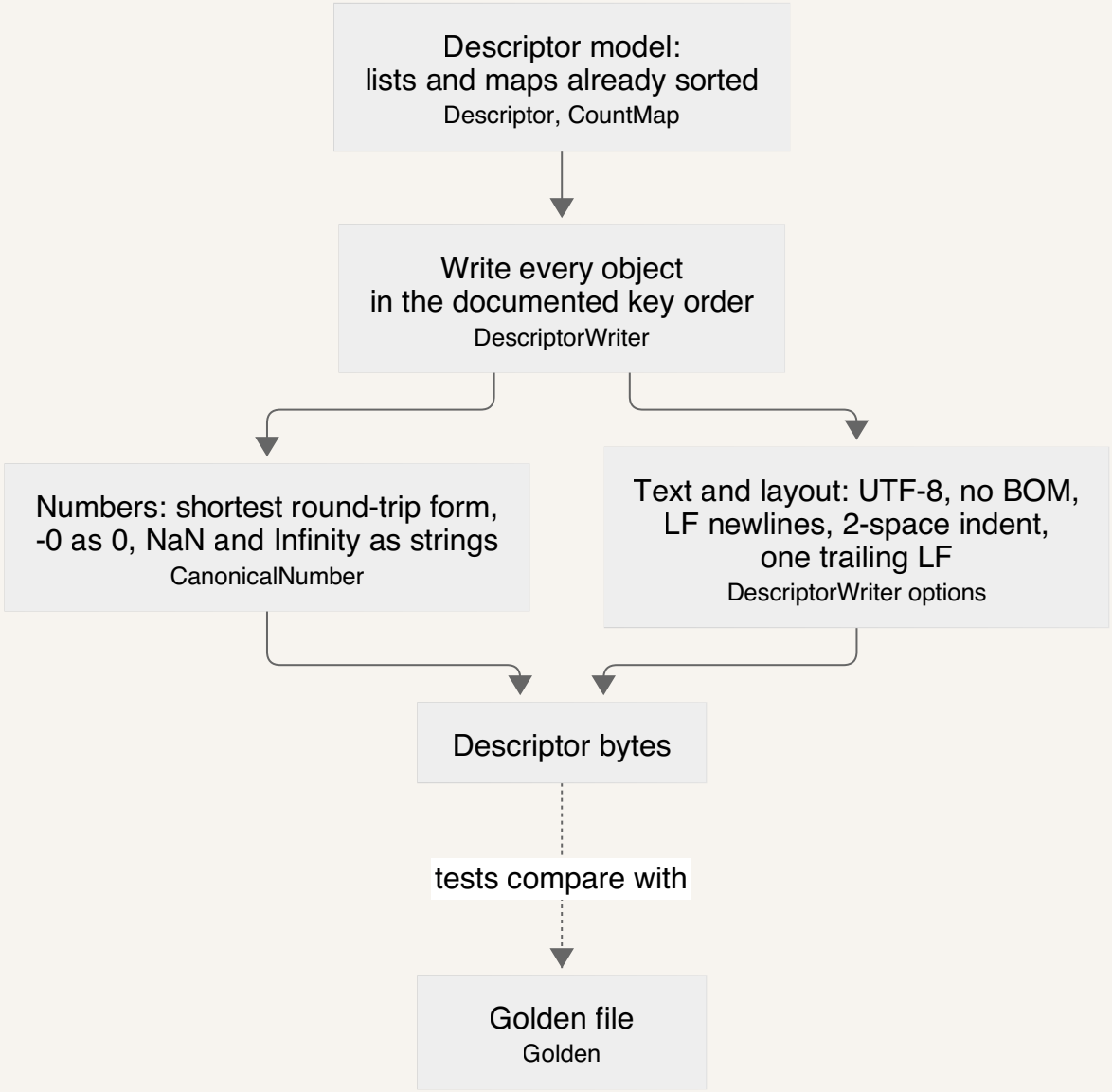



*Source:* `Fields/RowClassAccumulator.cs` *(planned).*

## 11. WRITING THE DESCRIPTOR

The finished descriptor is a tree of plain model objects. The writer turns it into bytes in one fixed way: it writes each object’s keys in the order the schema document lists, and it never sorts, because the model

already holds every list and map in order. Numbers and text go through fixed rules, so the same model gives the same bytes on every machine.

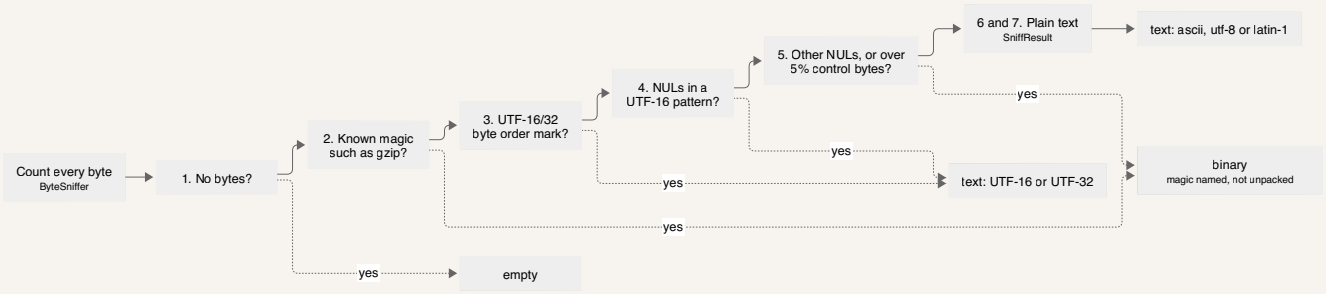



*Source:* `Descriptor/`, `Json/DescriptorWriter.cs`, `Json/CanonicalNumber.cs`; `../../tests/squalify.Tests/Golden.cs`. The key order is part of the schema: see descriptor.md.

## 12. TEXT, BINARY OR EMPTY?

The byte pass counts every byte of the file once and then applies seven rules, in order. The first rule that fits decides. A file is never rejected: odd bytes are counted, and invalid UTF-8 in ascii or utf-8 text is also reported as an issue. A few stray invalid bytes in an otherwise plain text file do not change its encoding,

because a single bad byte in a large log should not turn it into “Latin-1” or “binary”; the allowance is at most 8 per file and at most 1 per 64 KiB.




*Source:* `Sniff/ByteSniffer.cs`, `Sniff/SniffResult.cs`, `Sniff/MagicTable.cs`. The exact thresholds and the magic table are in descriptor.md.